

Mercer's Kernel function

The word “kernel” has many different meanings in mathematics, including density kernels (that you have studied for kernel density estimation), transition kernels of a Markov chain, and convolutional kernels (we will discuss in deep learning). Here, we will be considering a specific type of Kernel called Mercer's kernel.

The (Mercer's) kernel function $K : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ is a symmetric function such that for any set of N points and any choice of numbers $c_i \in \mathbb{R}$

$$\sum_{i=1}^N \sum_{j=1}^N K(\mathbf{x}^i, \mathbf{x}^j) c_i c_j \geq 0$$

Equivalent to matrix of kernel values (Gram) matrix being positive semi-definite.

Gram Matrix

Given N data points, and a kernel function we define a Gram Matrix as:

$$\mathbf{K} = \begin{pmatrix} \mathcal{K}(\mathbf{x}_1, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & & \\ \mathcal{K}(\mathbf{x}_N, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix}$$

Then ~~K~~ has to be positive semi-definite for
K to be called a Mercer's kernel.

Commonly Used Kernels

- Linear Kernel: $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$

$$\mathbf{x} \in \mathbb{R}^d$$

- Polynomial Kernel: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + 1)^p$ of degree p

- Radial Basis Function (RBF) or Gaussian Kernel:

$$\rightarrow K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2).$$

- Sigmoid Kernel: $K(x, y) = \tanh(\alpha x^T y + c)$

Let us consider 2-dimensional data

and $p=2$

$$K(\mathbf{x}, \mathbf{x}') = \left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}^T \begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix} + 1 \right)^2 = \left(x_1 x'_1 + x_2 x'_2 + 1 \right)^2$$

$$= \underline{x_1^2 x_1'^2} + \underline{x_2^2 x_2'^2} + \underline{x_1 x'_1} + \underline{x_2 x'_2}$$

Kernel embedding

Mercer's theorem: For every valid Kernel, there exists an embedding function $\phi(\mathbf{x})$ such that $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$

Example: ① Quadratic Kernel: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}')^2$.

polynomial kernel w/o the "+1" term

If original data is 2-d, then $K(\mathbf{x}, \mathbf{x}') = (x_1 x'_1 + x_2 x'_2)^2 = (x_1 x'_1)^2 + (x_2 x'_2)^2 + 2x_1 x'_1 x_2 x'_2$

$$\phi(\mathbf{x}) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]^T = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$$

② Polynomial kernel of degree 2 : $(\mathbf{x}^T \mathbf{x}' + 1)^2$

where

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \\ x_1 \\ x_2 \end{bmatrix}$$

Intuition

Any positive definite matrix $\mathbf{K}$ can be represented using an eigen decomposition of the form $\mathbf{K} = \mathbf{U}^\top \mathbf{\Lambda} \mathbf{U}$, where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues $\lambda_i > 0$, and $\mathbf{U}$ is a matrix containing the eigenvectors. Now consider element (i, j) of $\mathbf{K}$:

$$k_{ij} = \left(\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i} \right)^\top \left(\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:j} \right) \quad (1)$$

where $\mathbf{U}_{:i}$ is the i 'th column of $\mathbf{U}$. If we define

$$\phi(\mathbf{x}_i) = \mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i},$$

then we can write

$$k_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad (2)$$

This idea can be generalized to apply to kernel functions, not just kernel matrices; this result is known as Mercer's theorem.

Applications in Machine Learning

- Support Vector Machines (SVM)
- Kernel Principal Component Analysis (PCA)
- Gaussian Processes
- Radial Basis Function Networks (RBFN)

Support Vector Machines

Non-probabilistic models for classification of the form:

$$f(\mathbf{x}) = \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}, \mathbf{x}^i) + w_0$$

where $K(\mathbf{x}, \mathbf{x}^i)$ is a Mercer's kernel. Learning fixes values of α_i . These are chosen in such a way that only a few i s have non-zero α_i . These are called support vectors.

Assume $y_i \in \{-1, +1\}$, binary classification model.

Predicted class label = $\text{sign}(f(\mathbf{x}))$

$\hat{y}$

When Kernel is linear

If kernel is linear: $K(\mathbf{x}, \mathbf{x}^i) = \mathbf{x} \cdot \mathbf{x}^i$

Then the SVM classifier $f(\mathbf{x}) = \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}, \mathbf{x}^i) + w_0$ reduces to
 $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + w_0$ where $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$

Training objective in SVMs

Given a training dataset, find α_i s and w_0 so as to minimize a new margin loss called the Hinge loss.

Hinge loss: $L_H(y, f(\mathbf{x})) = \max(0, 1 - yf(\mathbf{x}))$

Loss is 0 iff $yf(\mathbf{x}) \geq 1$.

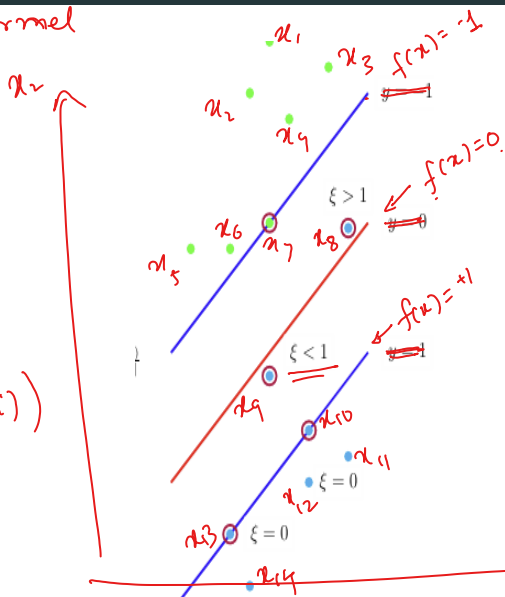
Understanding Hinge Loss geometrically

Assume linear kernel
 $d=2$

Points $x_1 \dots x_6$
 have $f(x_i) < -1$
 and $y_i = -1$

$\Rightarrow y_i f(x_i) > 1$

$\Rightarrow$ Loss: ξ_i
 $= \max(0, 1 - y_i f(x_i))$
 $= 0$



$$w_1 x_1 + w_2 x_2 + w_0 = 0$$

Consider x_8

$$f(x_8) < 0$$

$$\frac{y_8}{f(x_8)} = +1$$

$$\xi_8 = \max(0, 1 - y_8 f(x_8))$$

$$> 1$$

Consider x_9

$$\xi_9 =$$

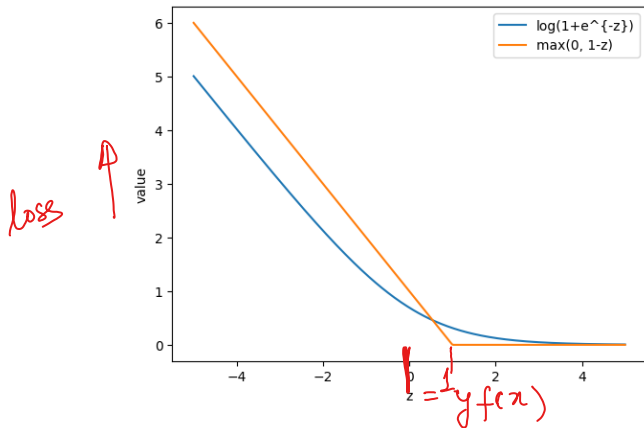
Comparing Hinge loss and logistic loss.

Advantage of hinge-loss: sparsity.

Comparing Hinge loss and logistic loss.

Advantage of hinge-loss: sparsity.

Binary logistic regression
 $\log(1 + e^{-yf(x)})$



Training SVMs

First consider the linear case:

Training objective:

$$L(w, w_0) = \min_{w, w_0} \left[C \sum_{i=1}^N \max(0, 1 - y_i(\underline{w \cdot x^i + w_0})) + \frac{1}{2} \|w\|^2 \right] \quad (3)$$

1) $L(w, w_0)$ is convex in w, w_0 .

$\max(0, g(w))$ where $g(w)$ is convex is always convex.

$\|w\|^2$ is convex

sum of convex functions is convex.

One option: Use sub-gradient descent algorithm.

Training objective as a constrained program.

1) Introduce new variable $\xi_1, \xi_2, \dots, \xi_N$
Equivalent to:

$$\begin{aligned} \min_{\mathbf{w}, w_0, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y^i (\mathbf{w} \cdot \mathbf{x}^i + w_0) \geq 1 - \xi_i \quad \forall i: 1 \dots N \\ & \xi_i \geq 0 \quad i: 1 \dots N \end{aligned} \tag{4}$$

- ① These two constraints imply that $\xi_i \geq \max(0, 1 - y^i f(x^i))$
- ② Since we minimize w.r.t ξ_i with positive $C > 0$
 $\xi_i = \max(0, 1 - y^i f(x^i))$

Solving the constrained optimization problem using Lagrangian dual

$$\min_{w, w_0, \xi_i} \frac{1}{2} \|w\|^2 + c \sum_{i=1}^N \xi_i$$

$$\text{s.t.} \rightarrow y_i \cdot (w x^i + w_0) \geq 1 - \xi_i \leftarrow \alpha_i$$

$$\xi_i \geq 0 \leftarrow \mu_i$$

$$\min_{w, w_0, \xi_i} \max_{\alpha_i, \mu_i} \left[\frac{1}{2} \|w\|^2 + c \sum_{i=1}^N \xi_i - \sum_{i=1}^N [y_i \cdot (w x^i + w_0) - 1 + \xi_i] \alpha_i - \sum_{i=1}^N \xi_i \mu_i \right]$$

$$L(w, w_0, \xi_i, \alpha_i, \mu_i)$$

General form of constrained opt.

$$\min_w F(w)$$

$$\text{s.t. } \underline{g_i(w)} \leq 0 \quad \forall i=1 \dots k$$

$$\min_w \max_{\alpha_i \geq 0} F(w) + \sum_{i=1}^k \alpha_i g_i(w)$$

Proof

KKT conditions for optimality:

$$\nabla_{w_0} [L(w, w_0, \xi_i, \alpha_i, \mu_i)] = 0$$

$$\Rightarrow - \sum_{i=1}^N \alpha_i^* y_i = 0$$

this implies that in the optimal solution $\sum_{i \in \text{positive examples}} \alpha_i^* = \sum_{i \in \text{negative examples}} \alpha_i^*$

$$\nabla_w F(w) + \sum_{i=1}^k \mu_i \nabla g_i(w) = 0$$

↖

$$\begin{aligned} \nabla_w [L(w, w_0, \xi_i, \alpha_i, \mu_i)] \\ w^* - \sum_{i=1}^N \alpha_i^* y_i x^i = 0 \\ \Rightarrow w^* = \sum_{i=1}^N \alpha_i^* y_i x^i \end{aligned}$$

Proof

$$\begin{aligned}
 & \frac{1}{2} \|w\|^2 + c \sum_{i=1}^N \zeta_i - \left(w \underbrace{\sum_{i=1}^N \alpha_i y_i x^i}_w - \underbrace{\sum_{i=1}^N \alpha_i y_i}_{=0} w_0 + \sum_{i=1}^N \alpha_i - \sum_{i=1}^N \alpha_i \zeta_i \right) \\
 & = \frac{1}{2} \|w\|^2 + \sum_{i=1}^N \alpha_i + \sum_{i=1}^N \zeta_i (c - (\alpha_i + u_i)) \\
 & = -\frac{1}{2} \left(\sum_{i=1}^N \alpha_i y_i x^i \right)^T \left(\sum_{i=1}^N \alpha_i y_i x^i \right) + \sum_{i=1}^N \alpha_i + \sum_{i=1}^N \zeta_i (c - (\alpha_i + u_i)) \\
 & = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x^i{}^T x^j + \sum_{i=1}^N \alpha_i + \sum_{i=1}^N \zeta_i (c - (\alpha_i + u_i))
 \end{aligned}$$